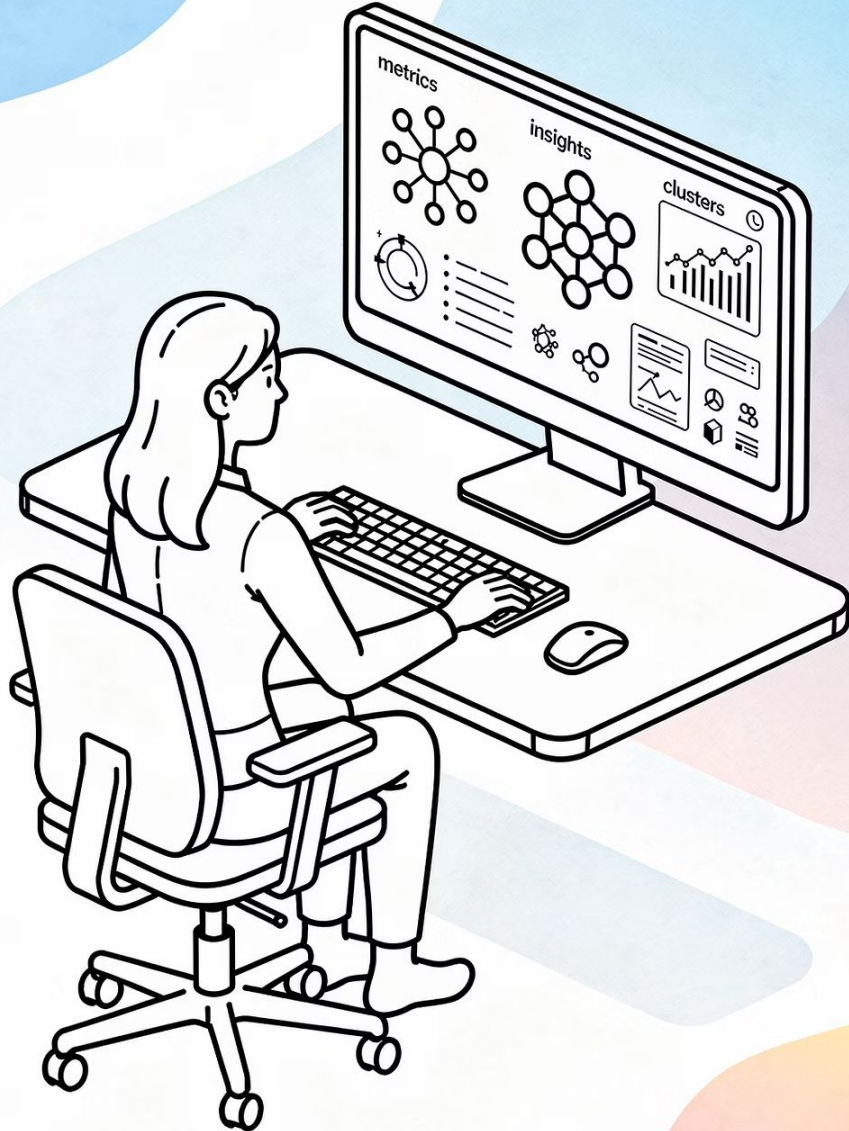




R · DALTOOLBOX

DATA MINING

Clustering: Didactic Examples

A guided learning sequence through unsupervised learning methods, organized by how each approach discovers structure in data. The central didactic focus: **there is no target variable during fitting** — what changes from one example to the next is the notion of similarity or grouping imposed by the method.

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br · eic.cefet-rj.br/~eogasawara

The Concept of Clustering

Clustering is an **unsupervised learning** technique that organizes observations into groups (clusters) — with no predefined target variable. The algorithm must discover structure on its own, guided only by a notion of similarity.

Intra-cluster

Maximize **similarity** among elements within the same group.

Inter-cluster

Minimize **similarity** between elements belonging to different groups.

Similarity Definition

Varies by method: **distance**, **density**, connectivity, and others — each shaping a different view of structure.

Key Characteristics of Clustering

Building on that definition, three core characteristics shape how clustering algorithms work — and what sets them apart from one another.

No Target Variable

The algorithm learns structure **exclusively from the input features**. There are no labels, no external supervision, and no response variable — the model discovers patterns entirely on its own.

Similarity Measures

Each method imposes a different notion of proximity: **K-means** uses Euclidean distance to a centroid, while **DBSCAN** uses regional density to define what it means to be "close."

Latent Structure

The goal is to **reveal hidden natural groupings** that emerge from the data's distribution — patterns that exist but were never explicitly labeled.

We will explore each of these characteristics in depth through practical, hands-on examples implemented in **R**.

Practical Guide: Clustering with daltoolbox in R

With the core concepts established, it's time to see clustering in action. In this section, we dive into hands-on R examples using the `daltoolbox` package — working through methods in a carefully structured sequence of increasing complexity.

The Methods We'll Explore

01

Partitional Methods

We begin with K-means and PAM — algorithms that partition data into k groups by minimizing intra-cluster variance. A natural first step for understanding cluster geometry.

02

Density-Based Structure

Building on partitional methods, we examine DBSCAN — an approach that defines clusters through regional density and automatically flags noise points that don't belong to any group.

03

Model Selection

Finally, we use `clu_tune` to systematically search the hyperparameter space and select the most robust, well-defined clustering solution for a given dataset.

Partitional Methods

cluster_kmeans – K-Means

K-Means is the most foundational partitional clustering algorithm. It is simple to implement, computationally efficient, and highly interpretable — making it the standard starting point for any unsupervised analysis. The method partitions data into **k non-overlapping groups** by iteratively minimizing intra-cluster variance, with no target variable required during fitting.

How the algorithm works

Initialize the centroids	Assign points to the nearest centroid	Update the centroids	Repeat until convergence
Randomly selects k initial points in the feature space as the starting centroid of each cluster. The choice of initialization can influence the final solution.	Each observation is assigned to the cluster whose centroid minimizes the Euclidean distance , forming k disjoint partitions of the dataset.	Each centroid is recalculated as the arithmetic mean of all observations currently assigned to that cluster, shifting it toward the cluster's center of mass.	Steps 2 and 3 are repeated until cluster assignments no longer change between iterations, indicating the algorithm has reached a stable local optimum.

Key Properties

Objective Function Minimizes the Within-Cluster Sum of Squares (WCSS) : $WCSS = \sum \sum \ x_i - \mu_k\ ^2$ where μ_k is the centroid of cluster k. Sensitive to scale — normalization is recommended.	Complexity Time complexity is $O(n \cdot k \cdot i \cdot d)$, where: n = number of observations k = number of clusters i = number of iterations d = number of dimensions Generally scales well for moderate datasets.	Assumptions Clusters are assumed to be convex, compact, and roughly equal in size . The algorithm uses spherical geometry (Euclidean distance), so it performs poorly on elongated, irregular, or density-varying cluster shapes.
--	---	--

 **Warning:** Results depend on the **random initialization** of centroids — different runs may produce different clusters. Additionally, the value of **k must be defined before training** and is not learned automatically by the algorithm.

  **References**
MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

Common Errors – K-Means

 **Warning:** Real labels (Species) are used *exclusively after* fitting, for pedagogical evaluation — **never during training**. Using them beforehand would constitute data leakage.

Wrong Choice of k

Defining k mechanically (e.g., always k=3) without considering the analytical purpose leads to meaningless clusters. Use the elbow method or indices such as silhouette to guide your choice.

Lack of Normalization

Features on different scales distort Euclidean distances, causing the algorithm to favor variables with larger magnitudes. Always normalize (Min-Max or Z-score) before applying K-Means.

Sensitivity to Outliers

Outliers shift centroids disproportionately, corrupting the clusters. Identify and handle outliers before clustering, or consider more robust algorithms such as K-Medoids.

Random Initialization

Random initial centroids may converge to local minima, producing unstable results. Use multiple initializations (n_init) or the **K-Means++** algorithm to reduce this risk.

cluster_kmeans – Configuration and Fitting

R Code

```
# install and load the package
# install.packages("daltoolbox")
library(daltoolbox)

# loading the iris dataset
# 150 observations, 4 numeric variables
data(iris)

# method configuration with k=3
# one cluster per expected species
model <- cluster_kmeans(k=3)

# model fitting and labeling
model <- fit(model, iris[,1:4])
clu <- cluster(model, iris[,1:4])

# distribution of points per group
table(clu)

## clu
## 1 2 3
## 96 33 21
```

Reading the code

- `cluster_kmeans(k=3)` — configures the model with 3 clusters (one per expected species)
- `fit(model, iris[,1:4])` — trains the model on the 4 numeric features (no labels used)
- `cluster(model, iris[,1:4])` — assigns each observation to a cluster
- `table(clu)` — shows the distribution of points across the 3 clusters

□ The value of `k=3` was chosen to match the known number of species — in practice, `k` must be determined analytically (e.g., elbow method, silhouette index).

cluster_kmeans – Evaluation and Normalization

PARTITIONAL METHODS

Key Concepts

External evaluation (pedagogical only)

After clustering, we can compare the resulting clusters with known labels (such as `iris$Species`) to interpret group quality. This is done *after* training — the actual labels **never participate** in the clustering process. The `evaluate()` function computes cluster entropy: the lower the value, the purer the groups.

Why normalize?

K-means uses Euclidean distance. If one variable has a much larger scale (e.g., centimeters vs. millimeters), it dominates the distance calculation and distorts the clusters. **Min-max** transformation places all variables on the same scale [0, 1], giving equal weight to each dimension.

 **Practical rule:** Always normalize before applying K-means when variables have different scales. The entropy dropping from 0.737 to 0.418 illustrates this gain.

Code – Evaluation

```
# --- External evaluation (post-clustering) ---
# The actual labels are used only for interpretation,
# they were not used during training.
eval <- evaluate(model, clu, iris$Species)
eval
## $clustering_entropy
## [1] 0.7375436
## high entropy = less pure groups
```

Code – Normalization & Re-evaluation

```
# --- Min-max normalization ---
# Places all variables on the [0, 1] scale
iris_mm <- transform(
  fit(minmax(), iris), iris)

# Re-fitting and labeling
model <- fit(model, iris_mm[,1:4])
clu <- cluster(model, iris_mm[,1:4])
table(clu)
## 1 2 3
## 39 50 61
## more balanced distribution

# Evaluation after normalization
eval2 <- evaluate(
  model, clu, iris_mm$Species)
## $clustering_entropy
## [1] 0.4177655
## lower entropy = purer groups
```

K-Means Results: Before and After Normalization

The following comparison demonstrates the **practical impact of feature scaling** on K-means clustering — showing how Min-Max normalization leads to more balanced and purer cluster assignments.

Without Normalization, k=3

Cluster Distribution: **96, 33, 21**

Clustering Entropy: **0.7375**

With Min-Max Normalization, k=3

Cluster Distribution: **39, 50, 61**

Clustering Entropy: **0.4178**

- ❏ **Why normalization is critical for K-means:** K-means relies on Euclidean distance — variables with larger scales dominate the distance calculation and skew cluster boundaries. Here, entropy dropped from **0.7375 to 0.4178** after normalization, producing more balanced and well-defined clusters. Always normalize before applying K-means when features have different scales.

cluster_pam – Partitioning Around Medoids

PAM is a robust alternative to K-Means that shares the same partitional logic — divide data into **k** non-overlapping clusters — but replaces computed centroids with **medoids**: actual data points that minimize total dissimilarity to all other cluster members. This makes PAM more **interpretable** and significantly more **resistant to outliers**, at the cost of higher computational complexity.

Medoids vs. Centroids

Centroid (K-Means)

Arithmetic mean of all points in the cluster. May **not correspond to any real observation** in the dataset. **Sensitive to outliers** — extreme values shift the mean, distorting the cluster center.

Medoid (PAM)

The actual data point that **minimizes total dissimilarity** to all other members of the cluster. Always a **real, observable data point**. **Robust to outliers** — extreme values have less influence on the selection of the most representative point.

Key Properties

Objective Function

Minimizes **Total Within-Cluster Dissimilarity** — the sum of distances from each point to its medoid. Compatible with **any distance metric**, not just Euclidean.

Complexity

$O(k \cdot (n-k)^2)$ per iteration — more computationally expensive than K-Means, but delivers more stable and robust cluster assignments.

When to Use

Noisy data or presence of **outliers**; when **interpretability** of the cluster center is essential; when features have different distance semantics.

Results on Iris Dataset

Cluster Distribution

clu: 1 2 3
50 62 38

Clustering Entropy

0.3939 — competitive with normalized K-Means, achieved **without any prior scale preprocessing**.

Reference

Kaufman, L., & Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley. <https://doi.org/10.1002/9780470316801>

cluster_pam – Partitioning Around Medoids

What is PAM?

PAM (Partitioning Around Medoids) is a clustering algorithm closely related to K-means, but with a fundamental distinction: rather than using centroid vectors computed as the mean of cluster points, PAM selects **medoids** — actual data points from the dataset — as cluster centers.

- **Similarity to K-means:** requires the number of clusters `k` as a parameter and follows the same fit-and-label workflow
- **Medoids vs. Centroids:** by anchoring clusters to real data points, PAM is **more robust to outliers**, since extreme values cannot artificially shift the cluster center
- **When to use:** datasets with noise, outliers, or whenever interpretability of the cluster center is essential

Step-by-step

- Initialize PAM with `cluster_pam(k=3)` — the same interface as K-means
- Fitting and labeling follow the same pipeline: `fit()` → `cluster()`
- Cluster distribution: 50, 62, and 38 points — cluster 1 is perfectly pure (entropy = 0)
- Clustering entropy: **0.394** — lower than K-means without normalization

📌 **Key concept:** PAM uses medoids — real data points — instead of computed centroids, making it inherently more robust to outliers. Reference: Kaufman & Rousseeuw (1990).

```
# install.packages("daltoolbox")
library(daltoolbox)

data(iris)

# configure PAM with k=3 clusters
# (same interface as K-means)
model <- cluster_pam(k=3)

# fit the model to the data
model <- fit(model, iris[,1:4])
```

```
# assign cluster labels
clu <- cluster(model, iris[,1:4])

# cluster size distribution
table(clu)
## clu
## 1 2 3
## 50 62 38

# external evaluation (clustering entropy)
eval <- evaluate(model, clu, iris$Species)
## $clustering_entropy
## [1] 0.3938863
```

Summary: Clustering with daltoolbox in R

What is Clustering?

Unsupervised learning technique that discovers hidden structure in data without a target variable. Three core characteristics: **no labels**, **similarity measures**, and **latent structure**.

K-Means (`cluster_kmeans`)

Partitions data into `k` groups by minimizing WCSS. Sensitive to scale — Min-Max normalization reduced entropy from **0.737** to **0.418**. Key lesson: always normalize and choose `k` analytically (elbow method, silhouette).

PAM (`cluster_pam`)

More robust alternative to K-Means: uses **medoids** (real data points) instead of centroids. Achieved entropy of **0.394** without normalization. Preferred when data has outliers or interpretability of cluster centers matters.

Consistent daltoolbox Workflow

All clustering methods follow the same pipeline: `configure` → `fit()` → `cluster()` → `evaluate()`. This uniformity makes it easy to swap methods and compare results systematically.

The examples in this guide used the **iris** dataset as a controlled benchmark — a pedagogical choice that allows direct comparison between methods using known ground-truth labels.

References & Additional Sources

Academic references and online resources cited throughout this clustering guide.

Online Resources



CRAN Package – daltoolbox

cran.r-project.org/web/packages/daltoolbox

DAL Toolbox – GitHub

github.com/cefet-rj-dal/daltoolbox

daltoolbox Clustering Examples – GitHub

github.com/cefet-rj-dal/daltoolbox-examples

Key Academic References

- Ogasawara, E., Sa, A., Castro, A., Santos, C., Salles, D., Lima, J., Tavares, L., Salles, R. & Saidy, V. (2026). daltoolbox: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. CRAN.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
- Kaufman, L., & Rousseeuw, P. J. (1990). Finding Groups in Data: An Introduction to Cluster Analysis. Wiley.
<https://doi.org/10.1002/9780470316801>